LAB 2: FIGMA-TO-CODE, LAYOUTS & RESPONSIVE DESIGN

Bridging the Gap between Figma Specifications and Modern Responsive CSS Grids

1. Learning Objectives

Welcome to Lab 2. In this hands-on laboratory session, you will learn how to systematically translate user interface designs created in Figma into semantic HTML5 code styled with advanced CSS layouts. This guide is student-facing and will walk you through reading figma properties, implementing CSS Flexbox and Grid layouts, and ensuring your code is beautifully responsive.

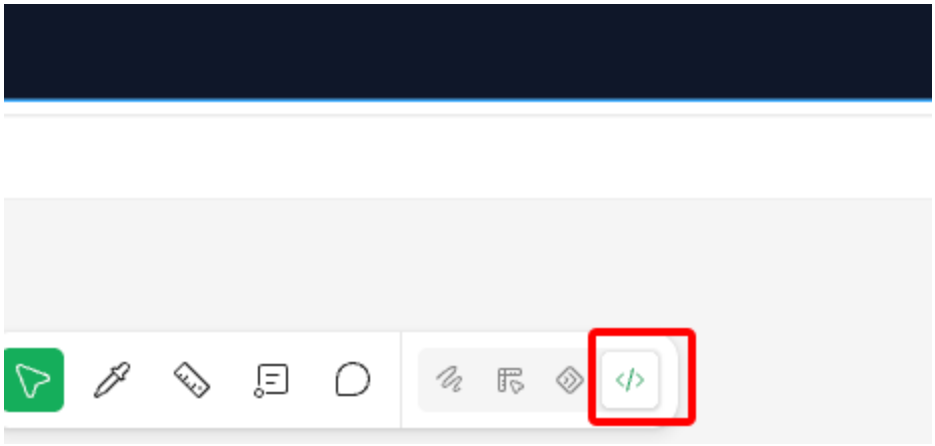
- Learn how to inspect Figma designs to pull spacing, fonts, sizing, colors, and export SVG/WebP assets.
- Master the technical differences, mechanics, and use cases of CSS Flexbox (1-Dimensional) and CSS Grid (2-Dimensional).
- Implement responsive web layouts using viewport meta settings, percentage sizing, and flexible breakpoints.
- Write media queries to adjust layout structures seamlessly between desktop, tablet, and mobile dimensions.
- Host and deploy a polished, multi-column responsive portfolio page using GitHub Pages.

2. Figma Inspection & Asset Extraction

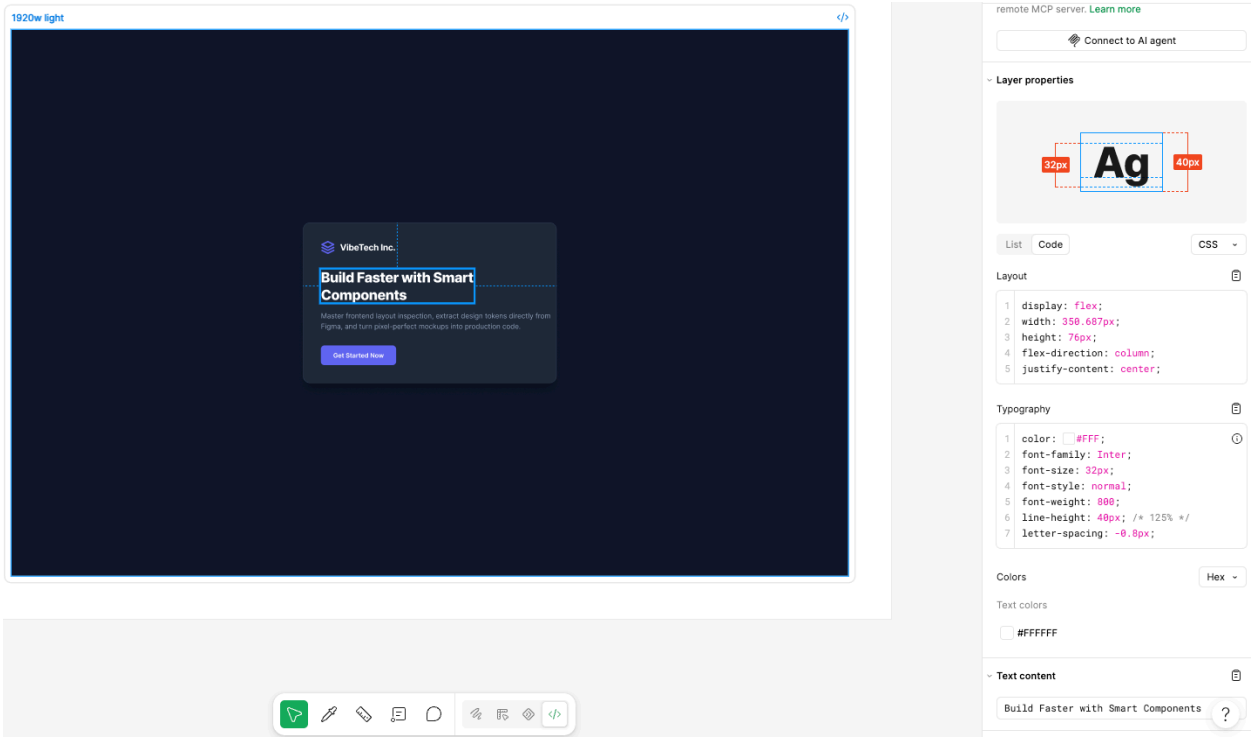
Figma is the industry-standard collaborative design tool. Before writing any CSS, you must understand how to read design rules directly from the canvas. Do not guess margins or font-sizes; read the exact specifications.

2.1. Inspecting Visual Specs

When you select an element on a Figma workspace using the Dev Mode or normal Inspect panel, look at the right sidebar:



- **Sizing & Layout:** Width (W) and Height (H) are provided in pixels (px). When translating to CSS, consider container constraints or relative values.
- **Colors:** Color codes are typically formatted in Hex (e.g., #2E75B6) or RGBA. Always set these as CSS variables to ensure code reusability.
- **Typography:** Font-family, Font-weight (e.g. SemiBold = 600), Font-size, and Line-height are critical. Line-height should be converted to unitless multipliers in CSS (e.g. line-height
- **Spacing:** Hover over an element and hold the Alt key (Option on Mac) while pointing to parent or sibling nodes to view exact pixel paddings and margins.



2.2. Exporting Vector vs Raster Assets

Figma design elements can be exported directly. Selecting the correct export format is crucial for website speed and scalability:

- **SVG (Scalable Vector Graphics):** Use this for logos, icons, and illustrations. SVG scales infinitely without losing sharpness and has minimal file sizes.
- **WebP:** Use this for photograph images (e.g. project screenshots, profile pictures). WebP provides superior lossless and lossy compression compared to PNG or JPG on the web.

> **Tip:** Always export vectors (logos/icons) as SVG. Never use PNG or JPG for vector logos, as they pixelate on high-resolution screens (Retina displays).

Example 1: Figma Inspection Challenge

Open the simple Hero Card Template. Inspect the primary Hero Card component:

1. Find the HEX code of the Primary Accent color and the Card Background color.

2. Find the font-family, font-size (in px or rem), and line-height of the main title heading.

3. Export the company logo in SVG format and save it inside an 'assets/' directory in your project.

3. Layout Architectures: CSS Flexbox vs. CSS Grid

Choosing between Flexbox and Grid is the first architectural decision you make when coding. While both layouts are highly powerful, they serve entirely different UI design patterns:

Feature	CSS Flexbox	CSS Grid
Dimensions	1-Dimensional (handles columns OR rows at a time).	2-Dimensional (handles columns AND rows simultaneously).
Design Approach	Content-First. Elements size themselves; the container aligns them.	Layout-First. The container defines a rigid grid, and elements occupy slots.

Ideal Use Cases	Navigation bars, simple lists, buttons groups, centered content cards.	Complex web pages, dashboard layouts, image galleries, responsive cards grids.

3.1. Standard Navigation Bar using Flexbox

Here is a common implementation of a utility navigation bar aligned horizontally. The logo is positioned on the left and the nav items on the right using Flexbox:

```
/* HTML Structure:
<header class="navbar">
  <div class="logo">MyBrand</div>
  <nav class="nav-links">
    <a href="#">Home</a>
    <a href="#">Projects</a>
    <a href="#">About</a>
  </nav>
</header> */

.navbar {
  display: flex;
  justify-content: space-between; /* Pushes child elements to the edge */
  align-items: center;           /* Centers vertically */
  padding: 1rem 2rem;
  background-color: #0f2537;
  color: white;
}

.nav-links {
  display: flex;
  gap: 1.5rem;                   /* Equal horizontal spacing between links */
}
```



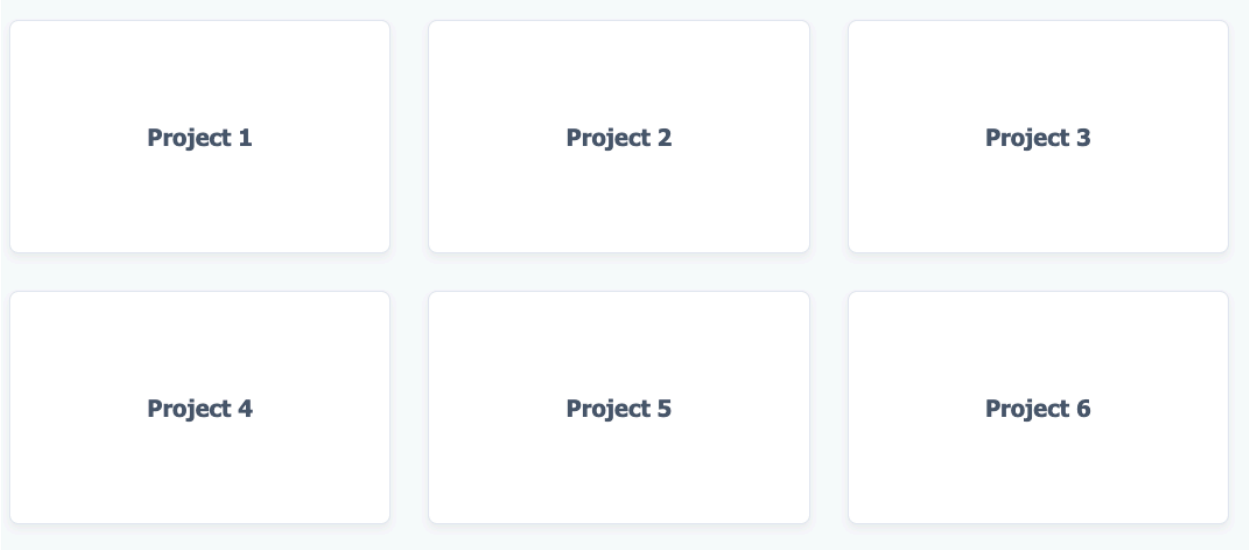
3.2. Responsive Card Grid Layout using CSS Grid

CSS Grid provides clean grid templates without requiring complex margin calculations. The following stylesheet establishes a fluid column layout that automatically wraps elements:

```
/* HTML Structure:
<section class="portfolio-grid">
  <div class="card">Project 1</div>
  <div class="card">Project 2</div>
  <div class="card">Project 3</div>
</section> */

.portfolio-grid {
  display: grid;
  /* repeat(auto-fit, minmax(300px, 1fr)) creates automatic grid columns */
  grid-template-columns: repeat(auto-fit, minmax(280px, 1fr));
  gap: 2rem;       /* Clears float margin hacks; sets spacing on rows and columns */
  padding: 2rem;
}

.card {
  background: white;
  border: 1px solid #e2e8f0;
  border-radius: 8px;
  box-shadow: 0 4px 6px rgba(0,0,0,0.05);
}
```



EXERCISE 1: Building a Two-Dimensional Image Gallery Layout

Write an HTML file with 4 item containers inside a parent container. Design a CSS Grid layout where the first item spans two column tracks while others adapt to default tracks. Apply a grid gap of 15px. Observe how CSS Grid controls both columns and rows seamlessly.

You can refer to the structure below:

```
<section class="exercise-grid">
    <div class="exercise-item item-1">Item 1 (Spans 2 columns)</div>
    <div class="exercise-item">Item 2</div>
    <div class="exercise-item">Item 3</div>
    <div class="exercise-item">Item 4</div>
</section>
```

4. Responsive Design & Media Queries

Modern websites must display correctly on devices ranging from small smartphone displays to ultra-wide desktop monitors. Responsive web design relies on flexible percentage-based layouts, responsive units (rem, em, vw, vh), and CSS Media Queries.

4.1. The Viewport Meta Tag

Without configuring the viewport metadata, mobile devices render pages using a standard virtual desktop width (often 980px). This shrinks the UI and makes text unreadable. This single line in the HTML <head> is non-negotiable:

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

4.2. Fluid Typography and Relative Units

Stop using pixel (px) sizing for text and containers. Relative sizing units are much safer:

- **rem (Root em):** Sets font sizes and margins relative to the root <html> font-size (usually 16px). 1.5rem equal 24px. This honors user accessibility font settings.
- **vw / vh (Viewport Width / Viewport Height):** Percentage relative to current browser screen size. 100vh fills the complete window height.

4.3. Writing Mobile-First Media Queries

Always write CSS styles with a mobile-first approach. Write the layout rules for mobile screens first, and layer screen-specific queries on top using min-width rules as the layout expands:

```
/* Default base styling for mobile screens (screens below 768px) */
.content-box {
  width: 100%;
  padding: 1rem;
}

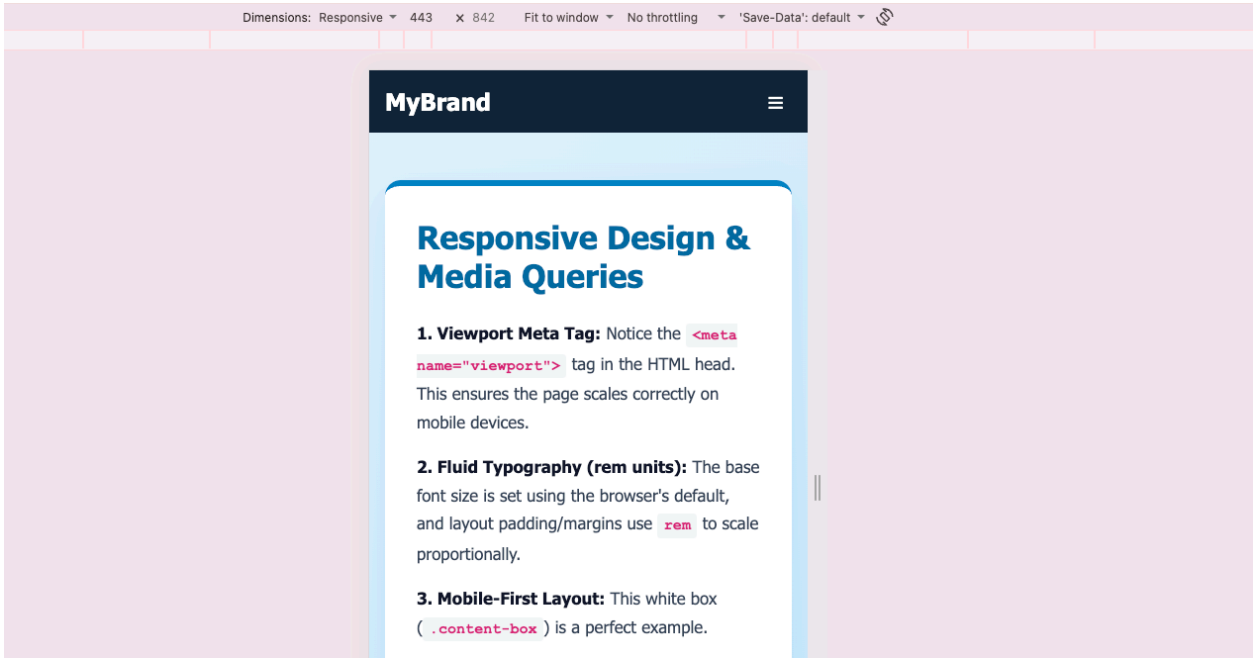
/* Tablet Device Breakpoint: Applies when screen is 768px wide or greater */
@media (min-width: 768px) {
  .content-box {
    width: 80%;
    padding: 1.5rem;
  }
}

/* Desktop Device Breakpoint: Applies when screen is 1024px wide or greater */
@media (min-width: 1024px) {
  .content-box {
    width: 60%;
    max-width: 1200px;
    padding: 2rem;
```

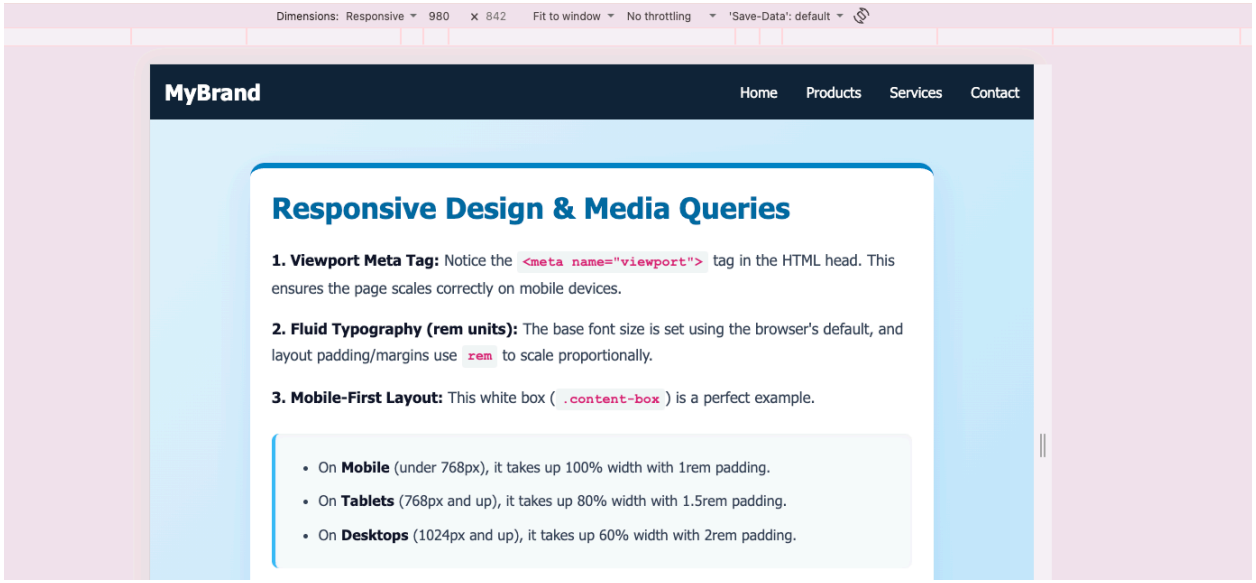
```
  }
}
```

For example:

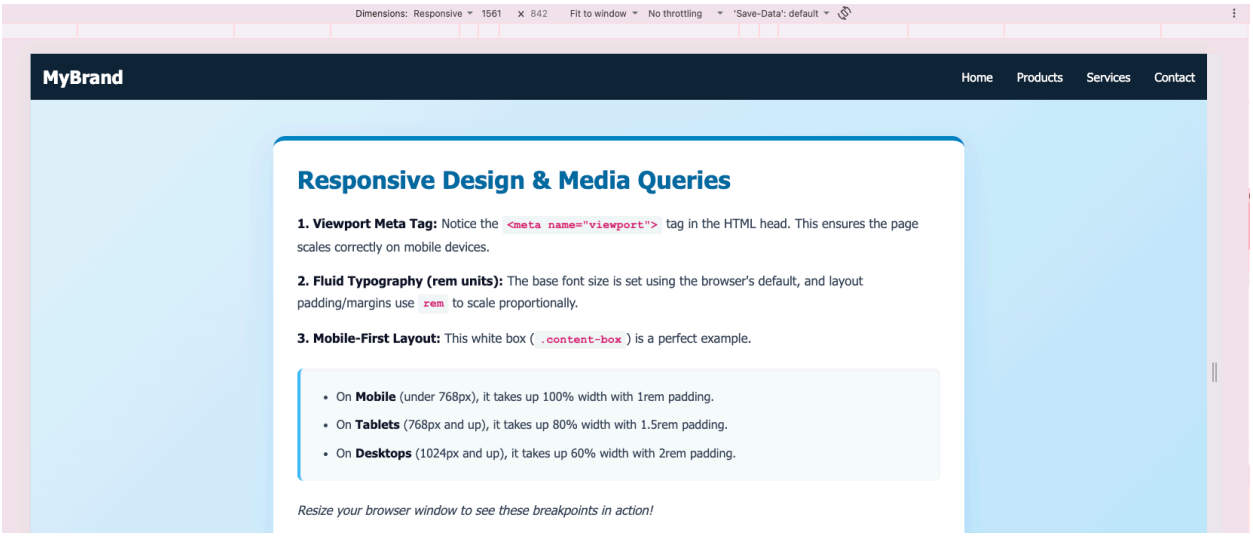
- Width < 768px:



- 768px < Width < 1024px:



- Width > 1024px:



EXERCISE 2: Interactive Mobile-First Navigation Menu

Write a media query that hides a desktop navigation horizontal bar (`display: none`) and displays a clean mobile menu button on screens under 768px wide. Add minor transition styles to toggle active menu configurations smooth and fluidly.

Use this JavaScript to toggle the mobile menu (put it in the body tag):

```
<script>
    const menuToggle = document.getElementById('menuToggle');
    const navMenu = document.getElementById('navMenu');

    menuToggle.addEventListener('click', () => {
        navMenu.classList.toggle('active');
    });
</script>
```

EXERCISE 3: Responsive Portfolio Gallery

Build a responsive portfolio grid listing 6 projects. It dynamically adapts: displaying 3 columns on desktops, 2 columns on tablets, and 1 column on smartphones.

Step 1: Code the HTML Structure

```
<!-- index.html -->
<!DOCTYPE html>
<html lang="en">
```

```
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Responsive Creative Portfolio</title>
    <link rel="stylesheet" href="styles.css">
</head>
<body>
    <header class="portfolio-header">
        <h1>Creative Portfolio</h1>
        <p>A showcase of clean layouts designed from Figma specifications</p>
    </header>

    <main class="grid-container">
        <!-- Project Card 1 -->
        <article class="project-card">
            <div class="card-image-box">
                <img src="https://picsum.photos/seed/project1/400/300" alt="Creative
Interface Project">
            </div>
            <div class="card-content">
                <h2>Creative Interface</h2>
                <p>Advanced Figma-to-code project focusing on responsive CSS
variables.</p>
                <a href="#" class="btn-more">View Project</a>
            </div>
        </article>

        <!-- Project Card 2 -->
        <article class="project-card">
            <div class="card-image-box">
                <img src="https://picsum.photos/seed/project2/400/300" alt="Dynamic
Application UI">
            </div>
            <div class="card-content">
                <h2>Dynamic Dashboard</h2>
                <p>Beautiful tabular layout featuring custom responsive borders and
```

```
grids.</p>
            <a href="#" class="btn-more">View Project</a>
          </div>
        </article>

        <!-- Bổ sung Card 3, 4, 5, 6 giống cấu trúc tương tự -->
    </main>
</body>
</html>
```

Step 2: Add CSS Variables & Setup Grid Layers

```
/* styles.css */
body {
    …
}

.portfolio-header {
    …
}

.portfolio-header h1 {
    …
}
```

Step 3: Establish Fluid Media Breakthrough Layout

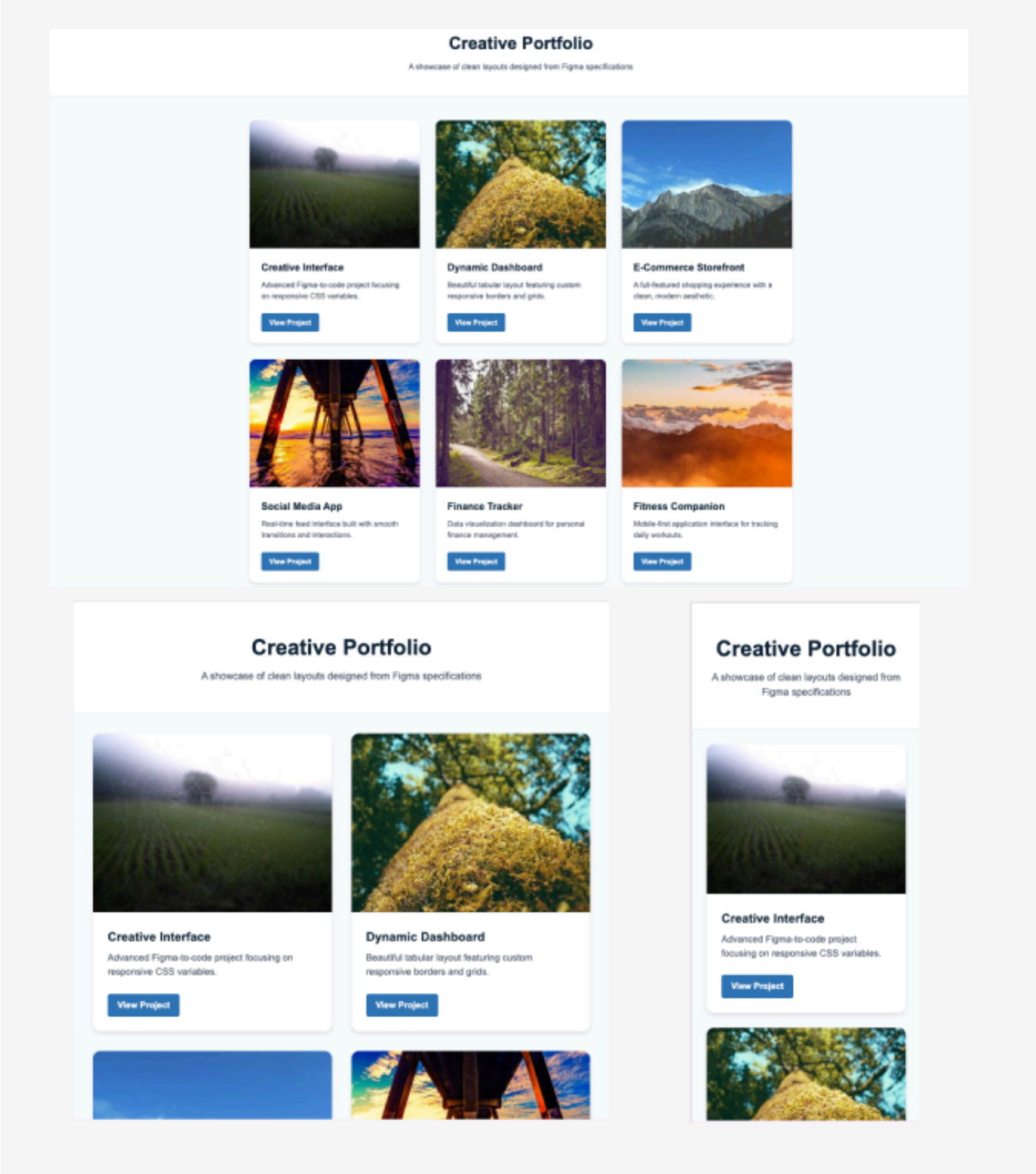
```
/* Card Grid Base Structure (Mobile Devices: 1 Column) */
.grid-container {
    …
}

.project-card {
    …
}
```

```
.project-card:hover {
    …
}

/* Tablet Layout (Screens 640px and Up: 2 Columns) */
@media (min-width: 640px) {
    .grid-container {
        …
    }
}

/* Desktop Layout (Screens 1024px and Up: 3 Columns) */
@media (min-width: 1024px) {
    .grid-container {
        …
    }
}
```



5. Homework Assignment: Responsive Landing Page

Build a complete website based on the provided Figma design and deploy it using GitHub Pages.

Functional Requirements:

1. **Follow the design:** Download images, icons, and colors directly from Figma.
2. **Navbar:** Use Flexbox for a sticky navbar with glassmorphism hover transitions.
3. **Grid Section:** Use CSS Grid to display at least 6 portfolio items.
4. **Responsive Design:** Must adapt to Mobile, Tablet, and Desktop.
5. **Hover States:** Add interactive hover effects to cards (e.g., image zoom, text fade-in).
6. **Deployment:** Deploy the final responsive page live to the Internet using GitHub Pages.

Figma design: [click here]

